

Homework 05

Name: Steven Jia

Date: September 27, 2026

Question 1: Estimate the bias and variance of KNN

I used seed **432051**. At the target point, the true mean is

$$f(0.5, 0.7, 1) = 0.5(0.5) + \sin(0.7) - 0.3(1)^2 = 0.5942177.$$

Table 1: KNN estimates and the 200-repetition error decomposition.

k	one_prediction	mean_prediction	squared_bias	variance	mse
1	0.7041	0.5608	0.0011	0.0352	0.0363
3	0.7275	0.5683	0.0007	0.0190	0.0197
5	0.7082	0.5615	0.0011	0.0154	0.0165
7	0.5822	0.5541	0.0016	0.0118	0.0134
9	0.5712	0.5482	0.0021	0.0112	0.0133
11	0.6066	0.5400	0.0029	0.0095	0.0124
13	0.5070	0.5320	0.0039	0.0097	0.0136
15	0.5013	0.5233	0.0050	0.0087	0.0137
17	0.5434	0.5130	0.0066	0.0079	0.0145
19	0.4950	0.5053	0.0079	0.0071	0.0150
21	0.4731	0.4990	0.0091	0.0069	0.0159
23	0.4670	0.4905	0.0108	0.0066	0.0174
25	0.4592	0.4844	0.0121	0.0068	0.0188
27	0.4537	0.4790	0.0133	0.0064	0.0197
29	0.4400	0.4739	0.0145	0.0064	0.0209

The biggest numerical difference between MSE and squared bias plus variance was 5.03e-17, so the two sides match apart from numerical rounding.

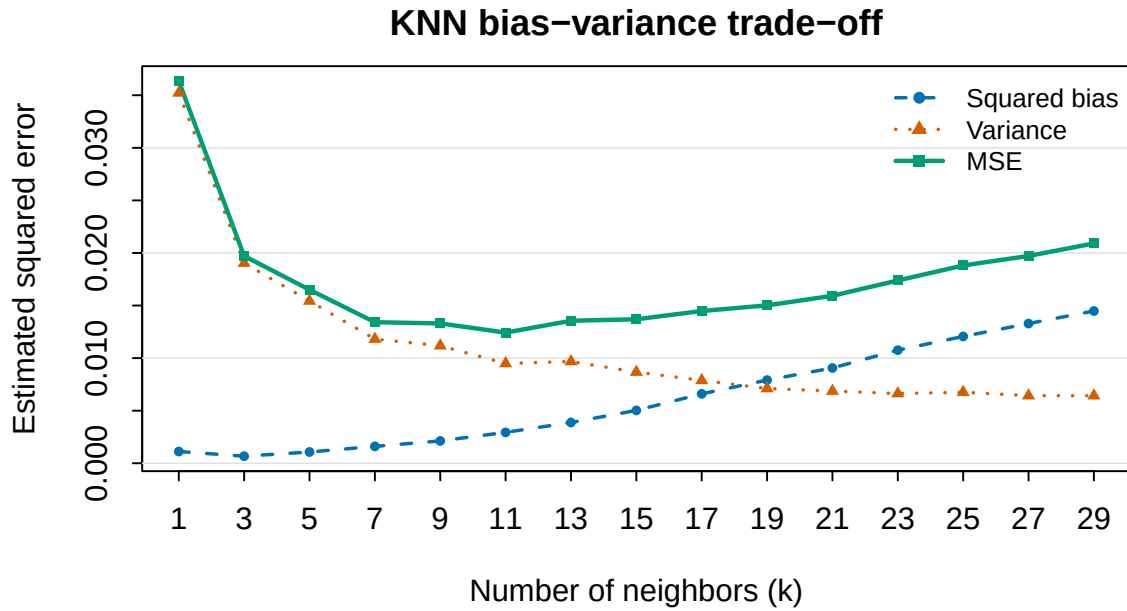


Figure 1: Estimated squared bias, variance, and MSE at the target point.

I can see the usual bias-variance trade-off in the plot. When k gets larger, the variance goes down because more responses are averaged. The squared bias goes up because some neighbors are farther from the target point. On this grid, the MSE is smallest at $k = 11$, where it is 0.0124. This is similar to increasing a ridge or lasso penalty: the model becomes less flexible, so variance can decrease while bias increases. My variance also includes changes in which observations become neighbors in each simulated data set, so it is not just $0.1^2/k$.

Question 2: Compare KNN with lasso

I used seed **432052**. I wrote the 5NN prediction and lasso coordinate-descent functions in R. Inside each cross-validation fold, I recalculated the predictor means, predictor scales, and response mean using only the training part of that fold.

Table 2: Test performance using the same data within each setting.

setting	method	test_mse	selected_lambda	nonzero_slopes
Independent	5NN	0.5396	NA	NA
Independent	Lasso	0.2208	0.0251	11
Correlated	5NN	0.1918	NA	NA
Correlated	Lasso	0.2464	0.0200	10

In my independent-predictor simulation, 5NN has test MSE 0.540, while lasso has 0.221. The 27 irrelevant variables still affect Euclidean distance, so they make it harder for KNN to find useful neighbors. Lasso can set many coefficients to zero, which helps in this case.

With correlated predictors, the 5NN MSE drops to 0.192. Since the variables move together, distance in all 30 variables contains more information about the important variables. Lasso still treats variables differently because the L_1 penalty can remove some of them. Also, the true model is nonlinear in X_2 and X_3 , while this lasso fit only uses linear terms. From these two simulations I cannot say that correlation always helps KNN, only that it helped in this example.

Question 3: Observed dimension and latent dimension

I used seed **43205**. I fit KNN with equal weights and Euclidean distance using all 100 observed X variables. I only used the latent variables to generate the data, not to fit KNN.

Table 3: One-dataset and 50-repetition average test MSE.

k	one_m3	one_m30	average_m3	average_m30
2	1.585	3.545	1.712	3.073
10	1.352	2.333	1.283	2.384
18	1.430	2.376	1.302	2.381
26	1.496	2.346	1.350	2.407
34	1.609	2.361	1.400	2.428
42	1.696	2.407	1.454	2.462
50	1.757	2.476	1.511	2.499
58	1.820	2.481	1.566	2.525
66	1.888	2.501	1.624	2.555
74	1.953	2.533	1.687	2.584
82	2.037	2.533	1.747	2.612

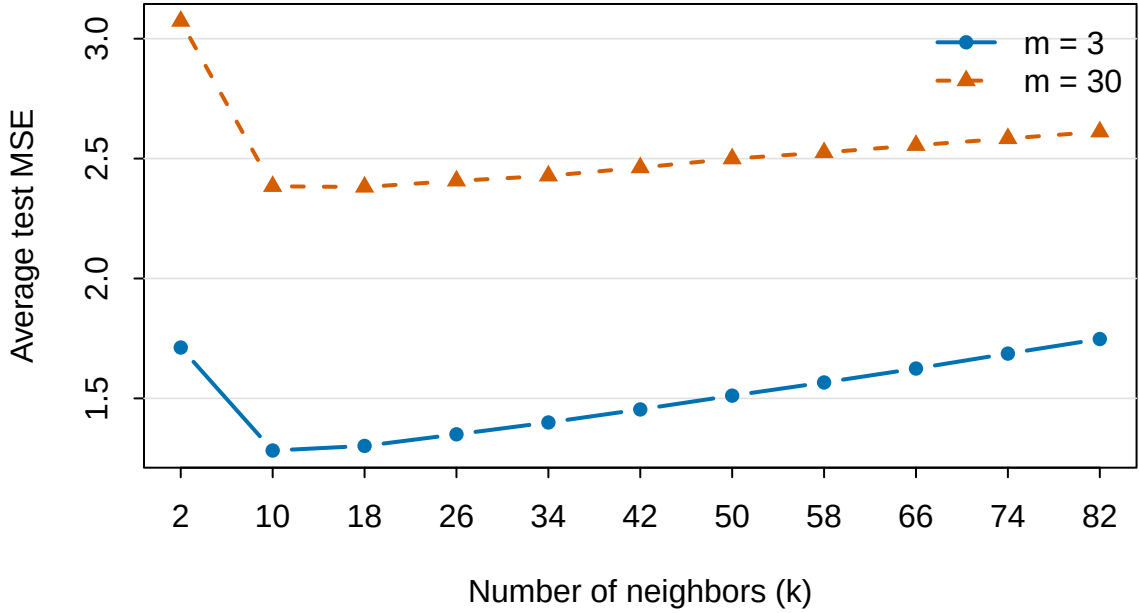


Figure 2: Average test MSE over 50 complete repetitions.

For $m = 3$, my smallest average MSE is 1.283 at $k = 10$. For $m = 30$, it is 2.381 at $k = 18$. These values come from the repeated simulation, not cross-validation. Both curves first go down because averaging reduces variance, and then go up because a large neighborhood adds bias.

My interpretation is that the two settings both have 100 observed variables, but their distance structures are different. When $m = 3$, being close in the observed variables is more likely to mean being close in the two latent variables that affect Y . When $m = 30$, the other 28 latent variables change the distances even though they do not affect Y . This makes the neighbors less useful. The noise η_i also changes all 100 observed variables. Conditional on $\mathbf{A}$,

$$\text{Cov}(\mathbf{X}_i | \mathbf{A}) = \mathbf{A}\mathbf{A}^\top + \mathbf{I}_{100},$$

This matrix is positive definite, so the observations still have full 100-dimensional support. They do not lie exactly in an m -dimensional linear subspace.

Question 4: Classify handwritten digits

I used the first 1,000 training rows and the first 500 test rows. My function calculates every distance directly. For distance ties I used the training row number, and `which.max` makes a

voting tie choose the smallest digit.

Table 4: 5NN classification errors.

distance	errors	test_error
Euclidean	61	0.122
Manhattan	69	0.138

Table 5: Euclidean confusion matrix (rows predicted, columns true).

	0	1	2	3	4	5	6	7	8	9
0	118	0	7	1	0	4	2	0	2	0
1	0	66	0	0	2	0	0	0	0	1
2	0	0	39	1	0	0	2	0	0	1
3	0	0	0	23	0	4	0	0	2	0
4	0	0	1	0	27	1	1	1	0	1
5	0	0	0	1	0	13	0	0	0	0
6	0	2	0	0	0	1	41	0	2	0
7	0	0	0	0	1	1	0	34	0	4
8	0	0	3	2	0	2	0	0	37	0
9	1	0	0	1	4	0	0	1	1	41

Table 6: Manhattan confusion matrix (rows predicted, columns true).

	0	1	2	3	4	5	6	7	8	9
0	118	0	6	1	0	5	3	0	4	0
1	0	67	2	0	2	0	1	0	0	1
2	0	0	35	0	0	0	2	0	0	0
3	0	0	1	24	0	4	0	0	2	0
4	0	0	1	1	27	1	0	1	0	1
5	0	0	0	1	0	11	0	0	0	0
6	0	1	0	0	0	2	40	0	2	0
7	0	0	2	1	1	0	0	34	0	5
8	0	0	2	0	0	3	0	0	34	0
9	1	0	1	1	4	0	0	1	2	41

With my code, Euclidean distance makes 61 errors (12.2%) and Manhattan distance makes 69 errors (13.8%). The most common specific mistake is a true 2 predicted as 0. This happens seven times with Euclidean distance and six times with Manhattan distance. Some other common pairs are 3/5, 4/9, and 7/9.

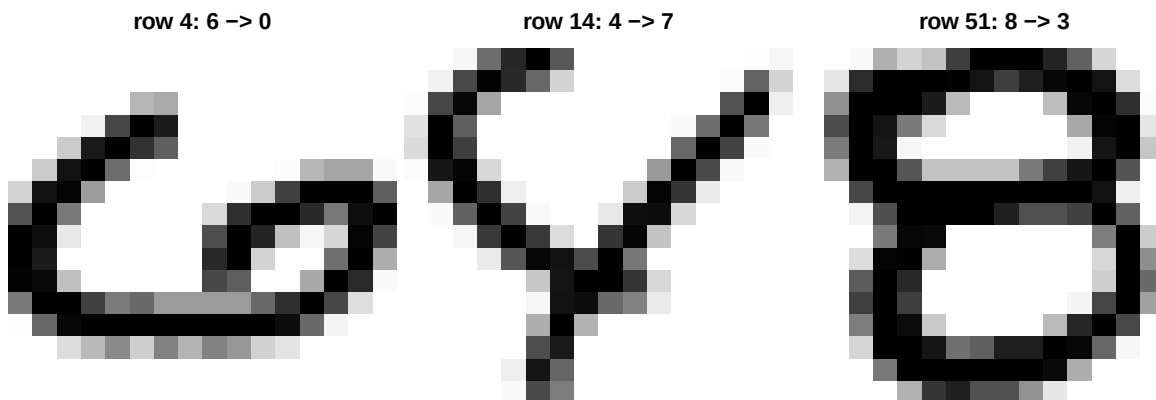


Figure 3: Three Euclidean 5NN errors; titles show test row, true label, and prediction.

The first mistake is the easiest one for me to understand. Its true label is 6, but the long stroke can look like 1 and the loop can look like 0. Because of this, a prediction of 0 is reasonable to me. The methods also do not miss exactly the same images. Their predictions differ on 22 out of 500 test images. Euclidean alone is wrong on 3 images, Manhattan alone is wrong on 11 images, and both are wrong with different labels on 8 images.

Question 5: Create a plot-formatting skill

I used a skill called `stat432-plot-polish` and installed it in my personal Codex skills folder. I made it explicit-only, then called `$stat432-plot-polish` on my Question 1 plot. The skill only changed the formatting. The three lines use the same values in both plots.

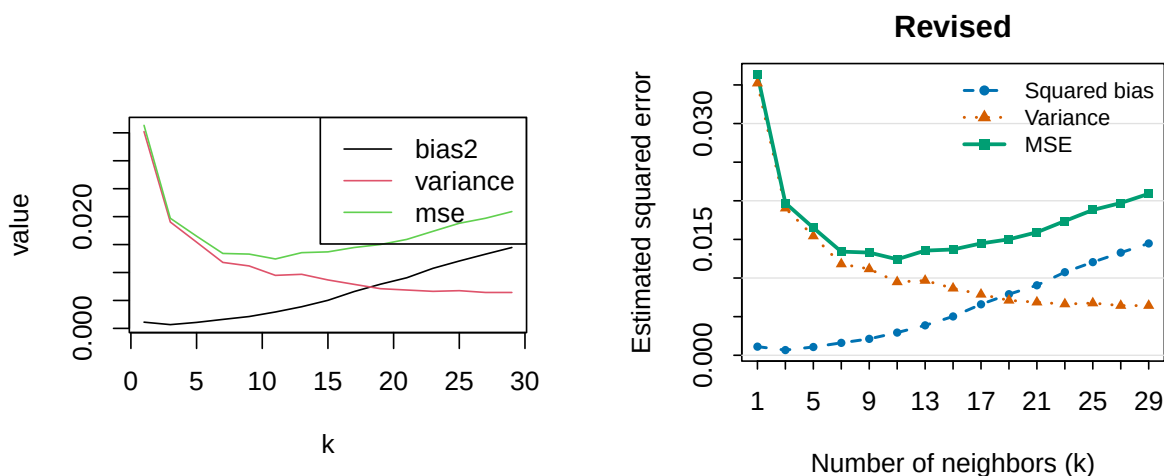


Figure 4: Original plot (left) and the skill-guided revision (right).

I like the revised plot more because the three lines are easier to tell apart. The larger points also make the small changes between nearby values of k more obvious. In the original plot, those changes are easier to miss because the lines look like one smooth overall shape. The rule I most want to keep is using different point shapes and readable point sizes. This was the change that helped me most in this plot.

Final SKILL.md:

```
---
name: stat432-plot-polish
description: Improve the formatting of STAT 432 homework plots in R or Python
only when the user explicitly asks to use this skill.
---

# STAT 432 Plot Polish

Improve readability without changing the data, statistical calculation,
plotted values, or substantive conclusion.

- Use balanced margins so titles, axes, legends, and annotations are not
clipped.
- Write a short, informative title and, when useful, a subtitle that states
the comparison without claiming a result the plot does not show.
- Label both axes with the quantity and units or scale; use meaningful tick
marks and avoid unnecessary decimals.
- Use direct series labels when practical; otherwise place a compact legend
where it does not cover the data.
- Choose a colorblind-friendly palette with clearly separated colors and also
distinguish overlapping series by line type or point shape.
- Start near a 6.5 by 4.2 inch single-panel figure with 11- to 12-point
labels; keep line widths and point sizes legible, then adjust for the final
page or multi-panel layout.
- Use a light, quiet background with subtle major grid lines; remove visual
elements that do not help interpretation.
- Preserve a consistent appearance across base R, ggplot2, matplotlib, or
seaborn while using the native controls of the chosen plotting system.

Before finishing, check the plot at its final rendered size and confirm that
every label is visible.
```